Memory Allocation

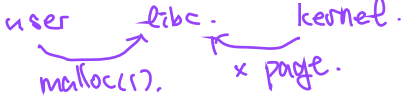
2026 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

Lecture 14
1.0.0

Based on original slides by Professor Jon Eyolfson.

How many of the following statements are true?

(1) In the x86 architecture, a memory reference that results in a TLB miss will also trigger a page fault.
(2) Every memory load and store instruction will trap into the OS kernel.
(3) On x86, the hardware (MMU) makes the page replacement decision.



How many of the following statements are true?

(1) The primary purpose of using a multilevel page table is to accelerate address translation.
(2) The primary purpose of a hardware TLB is to improve the speed of address translation.
(3) A multilevel page table reduces the TLB hit rate.
(4) A multilevel page table increases the TLB hit rate.

Your process uses a 2-level page table, and has a 75% TLB hit rate,
what is the closest expected memory access time?
Remember that TLB access time is much faster than memory access time and assume that it is instant.

TLB hit: 75% (1)

↑ data.

TLB miss 25% (1 (L1) + 1 (L0) + 1 (data).

= 0.75 + 0.25 × 3

= 1.5 × memory.

Static Allocation is the Simplest Strategy

Create a fixed size allocation in your program
e.g. `char buffer[4096];`

When the program loads, the kernel sets aside that memory for you

That memory exists as long as your process does, no need to free

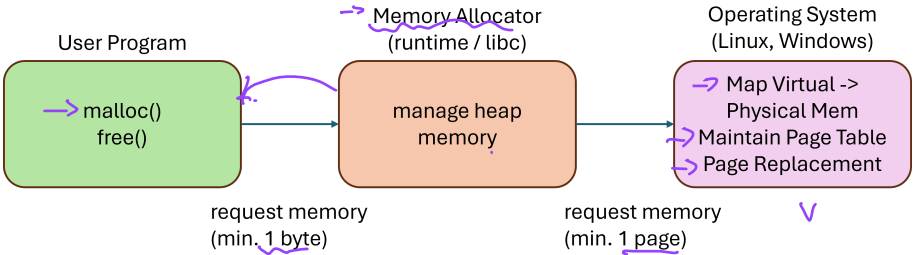
Dynamic Allocation is Often Required

You may only conditionally require memory
  Static allocations are sometimes wasteful

You may not know the size of the allocation
  Static allocations need to account for the maximum size (malloc // calloc).

Where do you allocate memory?
  You can either allocate on the stack or on the heap

What we have covered so far (dynamic).



Variable sized allocation

Given a block of memory, how do we allocate it to satisfy various memory allocation requests?

This problem must be solved in the C library

- Allocates one or more pages from kernel via brk/sbrk or mmap system calls
- Gives out smaller chunks to user programs via malloc

This problem also occurs in the kernel

- Kernel must allocate memory for its internal data structures

Variable sized allocation: headers

Consider a simple implementation of malloc

Every allocated chunk has a header with info like size of chunk

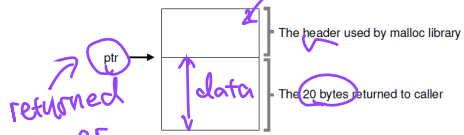


Figure 17.1: **An Allocated Region Plus Header**

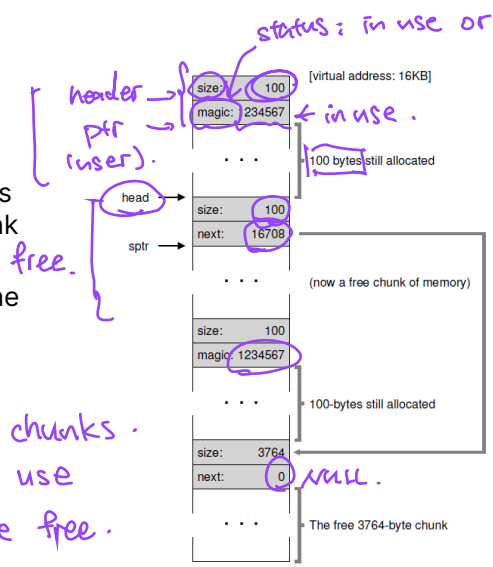
Free List

Free space is managed as a list

- Pointer to the next free chunk is embedded within the free chunk

⇒ The library tracks the head of the list

- Allocations happen from the head



Splitting and Coalescing

Suppose all the three chunks are freed (add linked list).

The list now has a bunch of free chunks that are adjacent

A smart algorithm would merge them all into a bigger free chunk

Must split and coalesce free chunks to satisfy variable sized requests

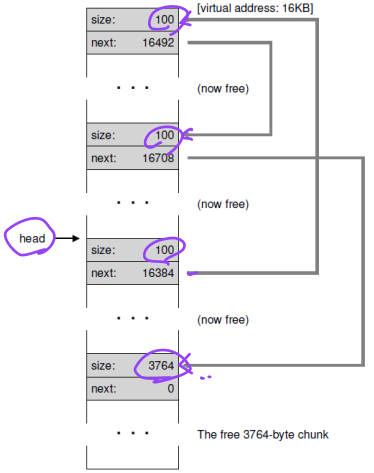


Figure 17.7: A Non-Coalesced Free List

Fragmentation is a Unique Issue for Dynamic Allocation

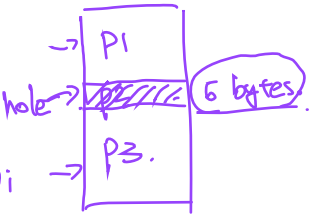
You allocate memory in different sized contiguous blocks
Compaction is not possible and every allocation decision is permanent

A fragment is a small contiguous block of memory that cannot handle an allocation
You can think of it as a "hole" in memory, wasting space

There are 3 requirements for fragmentation

1. Different allocation lifetimes
2. Different allocation sizes
3. Inability to relocate previous allocations (expensive).

malloc(100);
p1 =
p2 = malloc(5);
p3 = malloc(100);
free(p2);



There's Internal and External Fragmentation

External fragmentation occurs when you allocate different sized blocks
There's no room for an allocation between the blocks

Internal fragmentation occurs when you allocate fixed sized blocks
There's wasted space within a block (sitting w/ user).

Credit: Daniel Ritz

We Want to Minimize Fragmentation

Fragmentation is just wasted space, which we should prevent

We want to reduce the number of “holes” between blocks of memory
  If we have holes, we’d like to keep them as large as possible

Our goal is to keep allocating memory without wasting space

Allocator Implementations Usually Use a Free List

They keep track of free blocks of memory by chaining them together
  Implemented with a linked list

We need to be able to handle a request of any size

- For allocation, we choose a block large enough for the request
  Remove it from the free list
- For deallocation, we add the block back to the free list
  If it's adjacent to another free block, we can merge them

There are 3 General Heap Allocation Strategies

30 bytes...

- Best fit: choose the smallest block that can satisfy the request
  Needs to search through the whole list (unless there's an exact match)
- Worst fit: choose largest block (most leftover space)
  Also has to search through the list
- First fit: choose first block that can satisfy request

Allocating Using Best Fit (1)

Note that blocks with a blank background and a number are free

Where do we allocate this block?

Allocating Using Best Fit (2)

Where do we allocate this block?

Allocating Using Best Fit (3)

60 bytes available.

The next block does not fit anywhere

Allocating Using Worst Fit (1)

Where do we allocate this block?

Allocating Using Worst Fit (2)

Where do we allocate this block?

Allocating Using Worst Fit (3)

Next block fits exactly in remaining space

Best Fit and Worst Fit are Both Slow

Best fit: tends to leave very large holes and very small holes
Small holes may be useless

Worst fit: simulation says it's the worst in terms of storage utilization

First fit: tends to leave "average" size holes

The Buddy Allocator Restricts the Problem

Typically, allocation requests are of size 2^n
e.g. 2, 4, 8, 16, 32, ..., 4096, ...

Restrict allocations to be powers of 2 to enable a more efficient implementation
Split blocks into 2 until you can handle the request

We want to be able to do fast searching and merging

You Can Implement the Buddy Allocator Using Multiple Lists

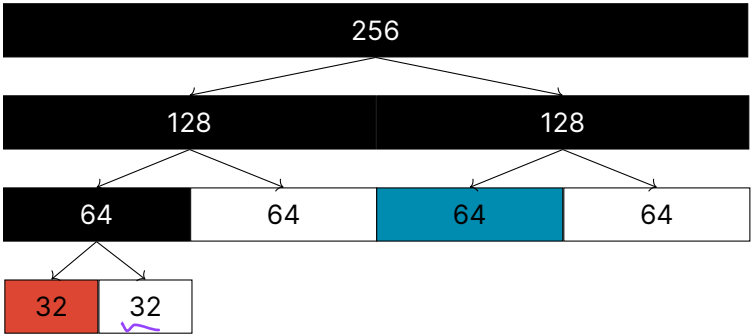
We restrict the requests to be 2^k, $0 \leq k \leq N$ (round up if needed)

Our implementation would use $N + 1$ free lists of blocks for each size

For a request of size 2^k, search the free list until we find a big enough block
- Search $k, k + 1, k + 2, \ldots$ until we find one
  - Recursively divide the block if needed until it's the correct size
  - Insert "buddy" blocks into free lists

For deallocations, we coalesce the buddy blocks back together
- Recursively coalesce the blocks if needed

Using the Buddy Allocator (1)

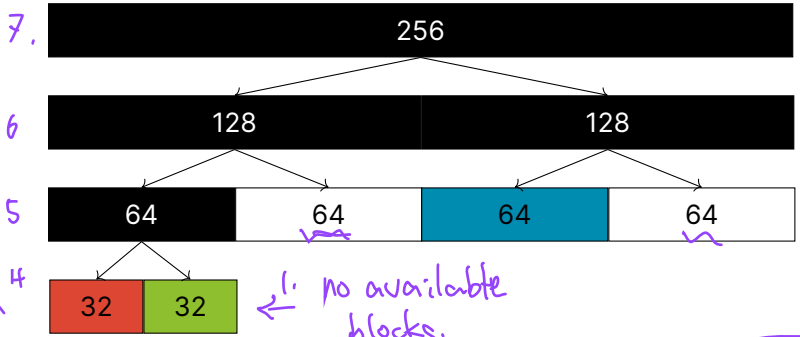


Where do we allocate a request of size 28?

round up 28 to the nearest k

k = 5 ⇒ 32 bytes.

Using the Buddy Allocator (2)



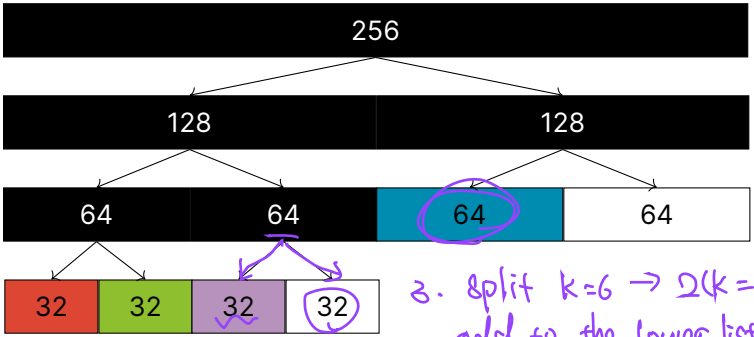
1. no available blocks.

2. k=6. available blocks found.

Where do we allocate a request of size 32?

32 ↑ k=5 = 32.

Using the Buddy Allocator (3)

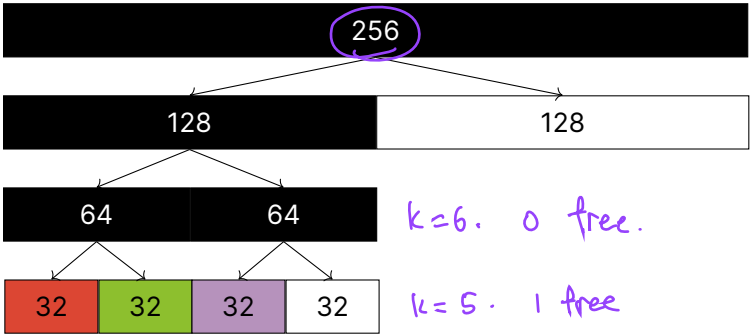


3. split k=6 → 2(k=5)
add to the lower list

What happens when we free the size 64 block?

4. use k=5 block
for allocation.

Using the Buddy Allocator (4)



Buddy Allocators are Used in Linux

Advantages

- Fast and simple compared to general dynamic memory allocation
- Avoids external fragmentation by keeping free physical pages contiguous

Disadvantages

- There's always internal fragmentation
  - We always round up the allocation size if it's not a power of 2

7 bytes ← 8 bytes.

maximum overhead

% of wasted space. due to internal fragmentation ?

Slab Allocators Take Advantage of Fixed Size Allocations

Allocate objects of same size from a dedicated pool
  All structures of the same type are the same size

Every object type has it's own pool with blocks of the correct size
  This prevents internal fragmentation

Slab is a Cache of “Slots”

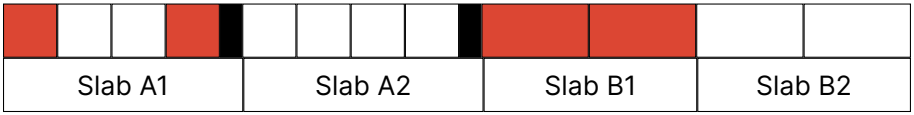
Each allocation size has a corresponding slab of slots
(one slot holds one allocation)

Instead of a linked list, we can use a bitmap
(there’s a mapping between bit and slot)
- For allocations, we set the bit and return the slot
- For deallocations, we just clear the bit

The slab can be implemented on top of the buddy allocator

Each Slab Can Be Allocated using the Buddy Allocator

Consider two object sizes: A and B



We can reduce internal fragmentation if Slabs are located adjacently
In this example A has internal fragmentation (dark box)